

COMPUTER VISION LAB 05: EXHAUSTIVE MASTER CHEAT-SHEET

Complete Theory, Mathematical Formulations, Algorithms, Code Implementations & OpenCV Parameter Reference

Course Code: AI-4002 | Target Manual: Lab 05 Manual (Image Segmentation) | Instructor: Talha Shahid

1. Core Theoretical Foundations & Mathematical Formulations

1.1 Introduction & Definition of Image Segmentation

Image Segmentation is the fundamental computer vision process of partitioning a digital image into distinct, non-overlapping regions or segments. Each segment corresponds to a meaningful object, anatomical structure, or region of interest in the scene. The ultimate goal is to simplify and transform a raw, high-dimensional pixel matrix into a semantically meaningful representation. Pixels belonging to the same segment share similar visual characteristics—such as color, intensity, texture, or gradient alignment—while adjacent segments differ significantly.

1.2 The 5 Mathematical Conditions of Image Segmentation

Formally, image segmentation partitions a complete image spatial domain R into n sub-regions $R_1, R_2, \dots, R_n$ such that the following five mathematical conditions are strictly satisfied:

- 1. Complete Coverage (Exhaustiveness):** The union of all segmented sub-regions must equal the entire image domain R : $\bigcup_{i=1..n} R_i = R$.
- 2. Connectivity:** Every segmented sub-region R_i is a connected set of spatial pixels.
- 3. Disjoint Segments (Mutually Exclusive):** Segments are strictly non-overlapping. The intersection of any two distinct sub-regions is an empty set: $R_i \cap R_j = \emptyset$ for all $i \neq j$.
- 4. Homogeneity Predicate:** All pixels within a single sub-region R_i satisfy a uniform logical property or predicate $P(R_i) = \text{TRUE}$ (e.g., pixel intensity variation within the region is below a threshold).
- 5. Boundary Distinctness:** Any two adjacent sub-regions R_i and R_j fail the homogeneity test when merged: $P(R_i \cup R_j) = \text{FALSE}$.

1.3 Mathematical Formulations Across Segmentation Approaches

Approach	Mathematical Formulation	Core Principle & Logic
Thresholding	$S(x,y) = \text{Object if } I(x,y) > T, \text{ else Background}$	Partitions an image based on comparing absolute pixel intensity against a scalar or local threshold T .
Edge Detection	$I = [I/x, I/y]^T$ $ I = \sqrt{(I/x)^2 + (I/y)^2}$	Identifies segment boundaries by locating local maxima in gradient magnitude (rapid rate of intensity change).
K-Means Clustering	$J = \sum_{j=1..K} \sum_{x_i \in C_j} x_i - \mu_j ^2$	Groups pixels in feature space (RGB/LAB) by minimizing the sum of squared Euclidean distances to cluster centroids μ_j .
Graph Theory	$\text{Cut}(A, B) = \sum_u \sum_v A(u,v) w(u,v)$	Models pixels as graph nodes and similarity as weighted edges; segments by finding minimum cost graph cuts.
Otsu's Variance	$\sigma_B^2(t) = w_0(t)w_1(t)[\mu_0(t) - \mu_1(t)]^2$	Maximizes inter-class variance between object class 0 and background class 1 across bimodal intensity histograms.

1.4 Real-Life Applications & Use-Case Matrix

Application Domain	Concrete Task & Target	Key Objective & Technical Challenge
Medical Imaging	MRI Brain Scan / CT Tumor Isolation	Isolates anatomical structures and pathological tumors in X-rays, CT, and MRI scans for diagnostic planning. Requires sub-pixel boundary precision.
Autonomous Vehicles	Real-Time Scene Parsing & Pedestrian Tracking	Segments road surfaces, lane markers, pedestrians, vehicles, and traffic signs in real-time video streams for obstacle avoidance.
Satellite Imagery	Land Cover & Urban Growth Classification	Classifies satellite images into distinct land types (forests, urban buildings, agricultural fields, water bodies) to monitor deforestation.
Biomedical Imaging	Microscopic Cell Nuclei Counting	Isolates individual cell boundaries and counts nuclei in fluorescence microscopy to diagnose hematological diseases.
Augmented Reality	Real-Time Object Overlay & Depth Alignment	Segments real-world foreground objects in camera feeds to correctly render 3D virtual overlays behind or in front of real objects.

2. Exhaustive Function & Parameter Breakdown Reference

Below is the complete, exhaustive breakdown of all OpenCV functions used in Lab 05 image segmentation pipelines, detailing every parameter, valid values, data types, and underlying mathematical mechanisms.

Function 1: cv2.threshold(src, thresh, maxval, type)

Applies global thresholding or automatic Otsu thresholding to a single-channel grayscale image array.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
src	ndarray (uint8 or float32)	Input single-channel grayscale image array.
thresh	float / int (0 - 255)	Threshold value. Pixels are compared against this value. Pass 0 when using cv2.THRESH_OTSU.
maxval	float / int (0 - 255)	Maximum value assigned to pixels that satisfy the threshold condition (typically 255).
type	OpenCV Flag	Thresholding type flag: • cv2.THRESH_BINARY: if $src(x,y) > thresh$ maxval, else 0 • cv2.THRESH_BINARY_INV: if $src(x,y) > thresh$ 0, else maxval • cv2.THRESH_TRUNC: if $src(x,y) > thresh$ thresh, else $src(x,y)$ • cv2.THRESH_TOZERO: if $src(x,y) > thresh$ $src(x,y)$, else 0 • cv2.THRESH_OTSU: Automatically calculates optimal threshold maximizing inter-class variance.

Return Value: Returns a tuple (retval, dst) where retval is the threshold used (float) and dst is the output binary array.

Function 2: cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)

Calculates dynamic local thresholds for small spatial neighborhoods to handle uneven lighting and shadows.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
src	ndarray (uint8)	Input 8-bit single-channel grayscale image array.
maxValue	float / int (255)	Non-zero value assigned to pixels satisfying the local threshold condition.
adaptiveMethod	OpenCV Flag	• cv2.ADAPTIVE_THRESH_MEAN_C: $T(x,y) = \text{Mean of blockSize} \times \text{blockSize neighborhood} - C$. • cv2.ADAPTIVE_THRESH_GAUSSIAN_C: $T(x,y) = \text{Gaussian weighted sum of neighborhood} - C$.
thresholdType	OpenCV Flag	Must be either cv2.THRESH_BINARY or cv2.THRESH_BINARY_INV .
blockSize	int (Odd integer > 1)	Size of the pixel neighborhood window used to calculate local threshold (e.g., 3, 5, 11, 21). Must be odd!
C	float / int	Constant subtracted from the calculated mean or Gaussian weighted sum (fine-tunes noise sensitivity).

Return Value: Returns the output binary image array dst of shape (H, W).

Function 3: cv2.inRange(src, lowerb, upperb)

Checks if array elements lie between upper and lower multi-channel bounds. Used for color-based segmentation in HSV space.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
src	ndarray (multi-channel)	Input array (typically 3-channel HSV or BGR image).
lowerb	ndarray / tuple (e.g. [H_min, S_min, V_min])	Inclusive lower boundary array/tuple for channel values.
upperb	ndarray / tuple (e.g. [H_max, S_max, V_max])	Inclusive upper boundary array/tuple for channel values.

Return Value: Returns a binary 8-bit single-channel mask array where pixels inside the range are set to 255, and 0 otherwise.

Function 4: cv2.Canny(image, threshold1, threshold2, apertureSize, L2gradient)

Finds structural boundaries using Canny's 5-stage optimal edge detection algorithm.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
image	ndarray (uint8)	Input 8-bit single-channel grayscale image.
threshold1	float / int (T_low)	First threshold for hysteresis. Weak edge candidates below threshold1 are discarded.
threshold2	float / int (T_high)	Second threshold for hysteresis. Edge candidates above threshold2 are marked as strong edges.
apertureSize	int (3, 5, or 7)	Aperture size for the Sobel operator used internally to compute gradients. Default is 3.
L2gradient	bool (True / False)	If True, computes gradient magnitude using L2-norm: $\sqrt{dx^2 + dy^2}$. If False, uses L1-norm: $ dx + dy $.

Return Value: Returns an 8-bit binary edge map image dst where edge pixels are 255.

Function 5: cv2.morphologyEx(src, op, kernel, iterations) & cv2.dilate()

Applies advanced morphological operations to refine binary masks, eliminate noise, or bridge gaps.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
src	ndarray (uint8 binary)	Input binary image mask array.
op	OpenCV Morph Flag	• cv2.MORPH_OPEN: Erosion followed by Dilation (removes background noise). • cv2.MORPH_CLOSE: Dilation followed by Erosion (closes small holes inside foreground). • cv2.MORPH_GRADIENT: Difference between dilation and erosion (extracts boundaries).
kernel	ndarray (np.uint8)	Structuring element matrix (e.g., np.ones((3,3), np.uint8)). Defines local neighborhood for operations.
iterations	int (≥ 1)	Number of times erosion/dilation is repeatedly applied.

Return Value: Returns processed binary mask dst of same size and type as src.

Function 6: cv2.distanceTransform(src, distanceType, maskSize)

Calculates the distance to the closest zero/background pixel for every non-zero foreground pixel.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
src	ndarray (8-bit binary)	Input 8-bit single-channel binary image where foreground objects are non-zero.
distanceType	OpenCV Flag	• cv2.DIST_L2: Exact Euclidean distance: $\sqrt{dx^2 + dy^2}$. • cv2.DIST_L1: City block / Manhattan distance: $dx + dy$. • cv2.DIST_C: Chessboard distance: $\max(dx , dy)$.
maskSize	int (3, 5, or cv2.DIST_MASK_PRECISE)	Size of the distance transform mask. 5 provides high precision for L2 metric.

Return Value: Returns a 32-bit floating-point image `dist` where each pixel value is the distance to the nearest zero pixel.

Function 7: cv2.connectedComponents(image, labels, connectivity, ltype)

Computes the connected components labeled image of a boolean or binary image.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
image	ndarray (8-bit binary)	Input 8-bit single-channel binary image to be labeled.
connectivity	int (4 or 8)	Pixel connectivity: 4-way (orthogonal neighbors) or 8-way (includes diagonal neighbors). Default is 8.
ltype	OpenCV Type Flag	Output label image type: cv2.CV_32S (32-bit signed integer) or cv2.CV_16U.

Return Value: Returns a tuple `(num\_labels, labels)` where `num\_labels` is the integer total count of unique labeled regions (including background label 0) and `labels` is the output array.

Function 8: cv2.watershed(image, markers)

Performs marker-based topographic watershed segmentation to separate touching or overlapping objects.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
image	ndarray (8-bit 3-channel)	Input 8-bit 3-channel color image array (BGR format).
markers	ndarray (int32 signed)	32-bit single-channel signed integer array of shape (H, W). Serves as both input seed markers and output map. • Seed Values: 1 = Background, 2..N = Object instances, 0 = Unknown transition regions. • Output Boundaries: Modified in-place; watershed boundary dams are set to -1.

Return Value: Modifies markers array in-place. Returns modified markers matrix where boundary lines are -1.

Function 9: cv2.kmeans(data, K, bestLabels, criteria, attempts, flags)

Finds centers of clusters and groups input samples around the clusters in feature space.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
data	ndarray (float32)	2D floating-point matrix of feature vectors of shape (N, dimensions). CRITICAL: Must be float32!
K	int (≥ 1)	Number of clusters to split the set by.
bestLabels	ndarray or None	Optional integer array storing initial cluster indices for each sample. Pass `None`.
criteria	tuple (type, max_iter, epsilon)	Termination criteria tuple: • cv2.TERM_CRITERIA_EPS: Stop if accuracy < epsilon. • cv2.TERM_CRITERIA_MAX_ITER: Stop after max_iter iterations. Combine via `cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER` (e.g., (type, 100, 0.2)).
attempts	int (e.g. 10)	Number of times the algorithm is executed using different initial labellings to find optimal compactness.
flags	OpenCV Flag	• cv2.KMEANS_RANDOM_CENTERS: Random initial centers. • cv2.KMEANS_PP_CENTERS: K-Means++ smart initialization.

Return Value: Returns `(compactness, labels, centers)` where `compactness` is sum of squared distances, `labels` is array of cluster indices, and `centers` is array of cluster centroid feature vectors.

3. Fully Annotated Code Implementations for Lab 05

Program 1: Thresholding Suite (Global, Adaptive, Otsu & HSV Color Masking)

```
# =====
# PROGRAM 1: THRESHOLDING SUITE (GLOBAL, ADAPTIVE, OTSU & COLOR MASKING)
# =====
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load grayscale and color images
gray_img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
color_img = cv2.imread('image.jpg') # BGR format

# A. Global Thresholding (Static T = 128)
# Parameters: src, thresh=128, maxval=255, type=THRESH_BINARY
ret1, global_mask = cv2.threshold(gray_img, 128, 255, cv2.THRESH_BINARY)

# B. Adaptive Gaussian Thresholding (Local window = 11x11, C = 2)
# Parameters: src, maxval=255, adaptiveMethod, thresholdType, blockSize=11, C=2
adaptive_mask = cv2.adaptiveThreshold(
    gray_img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2
)

# C. Otsu's Automatic Bimodal Thresholding (thresh=0, flags combined)
# Parameters: src, thresh=0, maxval=255, type=THRESH_BINARY + THRESH_OTSU
otsu_val, otsu_mask = cv2.threshold(gray_img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print("Calculated Otsu Threshold:", otsu_val)

# D. Color-Based Thresholding in HSV Space
# Convert BGR to HSV
hsv_img = cv2.cvtColor(color_img, cv2.COLOR_BGR2HSV)
# Define lower/upper hue bounds for target green color
lower_green = np.array([35, 50, 50], dtype=np.uint8)
upper_green = np.array([85, 255, 255], dtype=np.uint8)
# Parameters: src, lowerb, upperb
color_mask = cv2.inRange(hsv_img, lower_green, upper_green)
```

Program 2: Marker-Based Watershed Segmentation for Touching Objects

```
# =====
# PROGRAM 2: MARKER-BASED WATERSHED SEGMENTATION (TOUCHING OBJECTS)
# =====
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load color image and convert to Grayscale
image = cv2.imread('coins.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 2. Otsu thresholding to create inverted binary mask
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# 3. Morphological opening to eliminate small background noise
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)

# 4. Determine Sure Background area via dilation
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# 5. Determine Sure Foreground area via Distance Transform
# Parameters: src, distanceType=DIST_L2, maskSize=5
```

```

dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist_transform, 0.2 * dist_transform.max(), 255, 0)
sure_fg = np.uint8(sure_fg)

# 6. Find Unknown transition region
unknown = cv2.subtract(sure_bg, sure_fg)

# 7. Label connected component markers
# Parameters: image (uint8 binary)
_, markers = cv2.connectedComponents(sure_fg)

# SHIFT LABELS BY 1: Background becomes 1 (not 0)
markers = markers + 1

# Unknown regions set to 0 for watershed flooding
markers[unknown == 255] = 0

# 8. Apply Watershed (modifies markers array in-place)
# Parameters: image (8-bit 3-channel BGR), markers (32-bit signed int32)
cv2.watershed(image, markers)

# Mark watershed boundary dam lines (-1) in Red
image[markers == -1] = [0, 0, 255]

```

Program 3: K-Means Color Clustering Segmentation

```

# =====
# PROGRAM 3: K-MEANS COLOR CLUSTERING SEGMENTATION
# =====
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load input color image
image = cv2.imread('image.jpg')

# 2. Reshape from (H, W, 3) to 2D matrix of pixel vectors (N, 3)
pixel_values = image.reshape((-1, 3))

# CRITICAL STEP: Must explicitly convert pixel values to float32!
pixel_values = np.float32(pixel_values)

# 3. Define termination criteria
# Stop if 100 iterations reached OR epsilon change < 0.2
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
K = 4 # Number of color clusters

# 4. Apply cv2.kmeans
# Parameters: data (float32), K, bestLabels=None, criteria, attempts=10, flags
compactness, labels, centers = cv2.kmeans(
    pixel_values, K, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS
)

# 5. Convert center values back to 8-bit uint8 integers
centers = np.uint8(centers)

# 6. Map cluster labels to centroid color values
segmented_data = centers[labels.flatten()]

# 7. Reshape 1D array back to original 3D image shape (H, W, 3)
segmented_image = segmented_data.reshape(image.shape)

```

Program 4: Region Growing Segmentation (Stack-Based Traversal)

```
# =====
# PROGRAM 4: REGION GROWING SEGMENTATION (STACK-BASED TRAVERSAL)
# =====
import cv2
import numpy as np

def region_growing(img, seed, threshold=50):
    mask = np.zeros_like(img, dtype=np.uint8)
    stack = [seed] # Stack prevents recursion overflow
    seed_intensity = int(img[seed])

    while len(stack) > 0:
        x, y = stack.pop()
        if x < 0 or x >= img.shape[0] or y < 0 or y >= img.shape[1]:
            continue
        if mask[x, y] == 0:
            if abs(int(img[x, y]) - seed_intensity) < threshold:
                mask[x, y] = 255
                stack.extend([(x+1, y), (x-1, y), (x, y+1), (x, y-1)])
    return mask
```

4. Quick Reference & Exam Survival Kit

4.1 Master Comparison Matrix of Segmentation Techniques

Technique	Primary Input Data	Core Strengths	Primary Weakness / Limitation
Global Threshold	Grayscale Image	Ultra-fast, lightweight.	Fails completely under non-uniform illumination/shadows.
Adaptive Threshold	Grayscale Image	Handles uneven lighting & shadows.	Sensitive to local noise; requires window size tuning.
Otsu's Method	Bimodal Grayscale	Auto maximizes inter-class variance.	Fails on unimodal or heavily multimodal histograms.
Region Growing	Grayscale + Seed (x,y)	Clean connected region boundaries.	Highly sensitive to initial seed selection.
Watershed	Distance Transform + Markers	Superior separation of touching objects.	Prone to severe over-segmentation if markers are poor.
K-Means	2D Pixel Matrix (float32)	Works natively in multi-channel space.	Must specify K; ignores spatial pixel locations (x,y).

4.2 High-Yield MCQ Traps & Exam Trick Alerts

- **Trap 1 (K-Means Data Type Crash):** cv2.kmeans() WILL CRASH if the pixel matrix is uint8. You MUST explicitly convert pixels using pixel_values = np.float32(pixel_values)!
- **Trap 2 (Otsu's Flag Combination):** Otsu's thresholding requires combining flags: cv2.THRESH_BINARY + cv2.THRESH_OTSU and setting the initial threshold parameter thresh = 0.
- **Trap 3 (Watershed Boundary Label Value):** Watershed boundary dam lines between segment basins are assigned an integer label of -1 in the markers matrix.
- **Trap 4 (Distance Transform Input):** cv2.distanceTransform requires an 8-bit binary image input where target object pixels are non-zero (foreground).
- **Trap 5 (Watershed Marker Offset):** When preparing markers for Watershed using cv2.connectedComponents, you MUST add 1 (markers = markers + 1) so background becomes label 1, and set unknown transition regions to 0!